



**Tecnológico
de Monterrey**

**MA2003B Aplicación de Métodos Multivariados
en Ciencia de Datos**

Módulo 2: Relaciones de Dependencia

Profesor: Raul Gomez
Correo: rgomez@tec.mx
Oficina: A4-123A

Tabla de contenidos

I	Análisis de Regresión	3
1.	Método de mínimos cuadrados	5

Parte I

Análisis de Regresión

Capítulo 1

Método de mínimos cuadrados

Supongamos que tenemos un conjunto de datos

$$((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$$

Por ejemplo, consideremos el siguiente conjunto de datos simulado:

```
library(tidyverse)
library(krulRutils)
set.seed(123)
n <- 50
x_data <- runif(n, min = 0, max = 10)
epsilon_data <- rnorm(n, mean = 0, sd = 1)
y_data <- x_data + epsilon_data

sample_data_tbl <- tibble(
  x = x_data,
  y = y_data
)
```

Notemos que podemos visualizar estos datos mediante un diagrama de dispersión:

```
sample_data_plot <- sample_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  coord_fixed() +
  labs(
    title = "",
    x = "",
    y = ""
  )
```

```
) +  
theme_krul()  
  
sample_data_plot
```

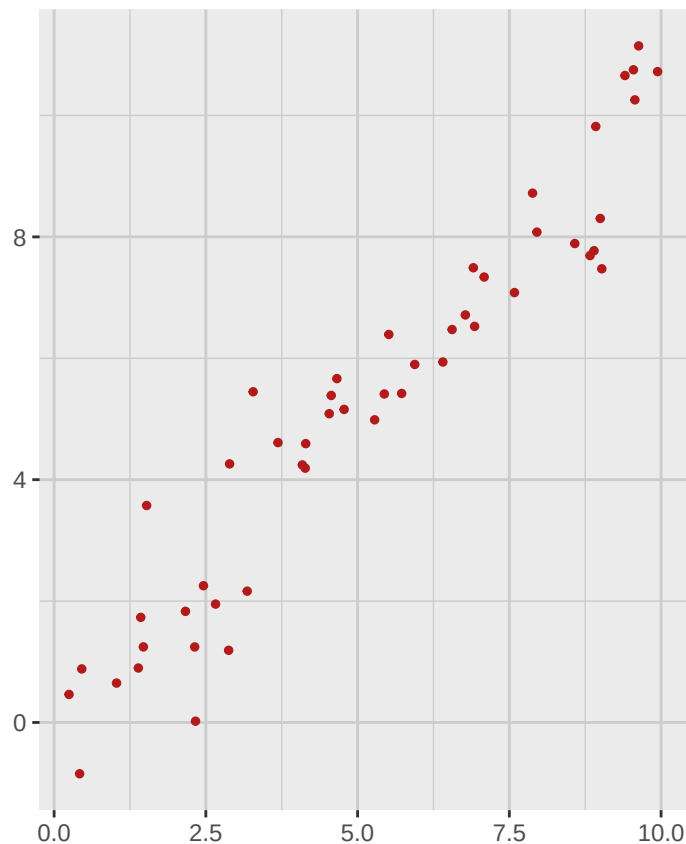


Figura 1.1: Diagrama de dispersión de los datos simulados.

Buscamos ahora generar una línea recta que mejor se ajuste a estos datos. Hay varias formas de definir “mejor ajuste”, pero una de las más comunes es minimizar la suma de los errores al cuadrado (SSE por sus siglas en inglés) entre los valores observados y los valores correspondientes sobre la línea recta propuesta. De manera más precisa, buscamos una recta

$$Y = \hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X$$

tal que si definimos $\hat{y}_i = \hat{f}(x_i)$ entonces minimicemos la suma

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Para encontrar la fórmula para $\hat{\beta}_0$ y $\hat{\beta}_1$ en términos de nuestros datos, utilizaremos un poco de álgebra lineal. Empecemos por fijar el *vector de las observaciones independientes*:

$$x = (x_1, x_2, \dots, x_n)$$

y el vector de las observaciones dependientes:

$$y = (y_1, y_2, \dots, y_n)$$

Consideraremos ahora el subespacio

$$W = \{\beta_0 1_n + \beta_1 x \mid \beta_0, \beta_1 \in \mathbb{R}\} \subset \mathbb{R}^n$$

donde $1_n = (1, 1, \dots, 1)$ es el vector en $\mathbb{R}^n$ con todas sus entradas iguales a 1. Notemos que W es un subespacio de dimensión 2 generado por los vectores 1_n y x . Nuestro objetivo es encontrar el vector $\hat{y} \in W$ que esté lo más cercano posible al vector de observaciones dependientes y . En otras palabras, buscamos minimizar el cuadrado de la distancia entre los vectores y e $\hat{y}$:

$$\|y - \hat{y}\|^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Para lograr esto, utilizaremos la proyección ortogonal. Recordemos que la proyección ortogonal de un vector v sobre un subespacio W es el vector en W que está más cercano a v . En nuestro caso queremos proyectar el vector de observaciones y sobre el subespacio W . Para ello primero necesitamos encontrar una base ortogonal para W , por lo que utilizaremos el proceso de Gram-Schmidt para ortogonalizar los vectores 1_n y x . Definimos entonces los vectores:

$$\begin{aligned} u_1 &= 1_n \\ u_2 &= x - \frac{\langle x, u_1 \rangle}{\langle u_1, u_1 \rangle} u_1 = x - \bar{x} 1_n \end{aligned}$$

donde

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

es la media de las observaciones independientes. Como u_1 y u_2 son ortogonales, podemos calcular la proyección ortogonal de y sobre W utilizando la fórmula:

$$\hat{y} = \frac{\langle y, u_1 \rangle}{\langle u_1, u_1 \rangle} u_1 + \frac{\langle y, u_2 \rangle}{\langle u_2, u_2 \rangle} u_2$$

Desarrollando esta expresión, obtenemos:

$$\hat{y} = \bar{y} 1_n + \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} (x - \bar{x} 1_n)$$

donde

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

es la media de las observaciones dependientes. Finalmente, al comparar esta expresión con la forma de la recta $y = \hat{\beta}_0 + \hat{\beta}_1 x$, podemos identificar los coeficientes de la siguiente manera:

$$\begin{aligned} \hat{\beta}_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \\ \hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{x} \end{aligned}$$

Observación 1.0.1. Recordemos que la covarianza muestral entre las observaciones x e y está dada por:

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

y la varianza muestral de las observaciones x está dada por:

$$s_x^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Por lo tanto, podemos reescribir el coeficiente $\hat{\beta}_1$ como:

$$\hat{\beta}_1 = \frac{s_{xy}}{s_x^2}$$

Para el ejemplo de datos simulados que presentamos al inicio de esta sección, podemos calcular la línea recta que mejor se ajusta a estos datos utilizando la función de regresión lineal en R:

```
fitted_data_tbl <- sample_data_tbl %>%  
  mutate(  
    fitted = fitted(lm(y ~ x, data = .))  
  )  
  
fitted_data_plot <- fitted_data_tbl %>%  
  ggplot(aes(x = x, y = y)) +  
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +  
  geom_segment(  
    aes(xend = x, yend = fitted),  
    color = c_pal("C green"),  
    alpha = 1  
  ) +  
  geom_point(color = c_pal("C red"), size = 1, alpha = 1) +  
  geom_point(  
    aes(x = x, y = fitted),  
    color = c_pal("C red"),  
    size = 1,  
    alpha = 1  
  ) +  
  coord_fixed() +  
  labs(  
    title = "",  
    x = "",  
    y = ""  
  ) +  
  theme_krul()  
  
fitted_data_plot
```

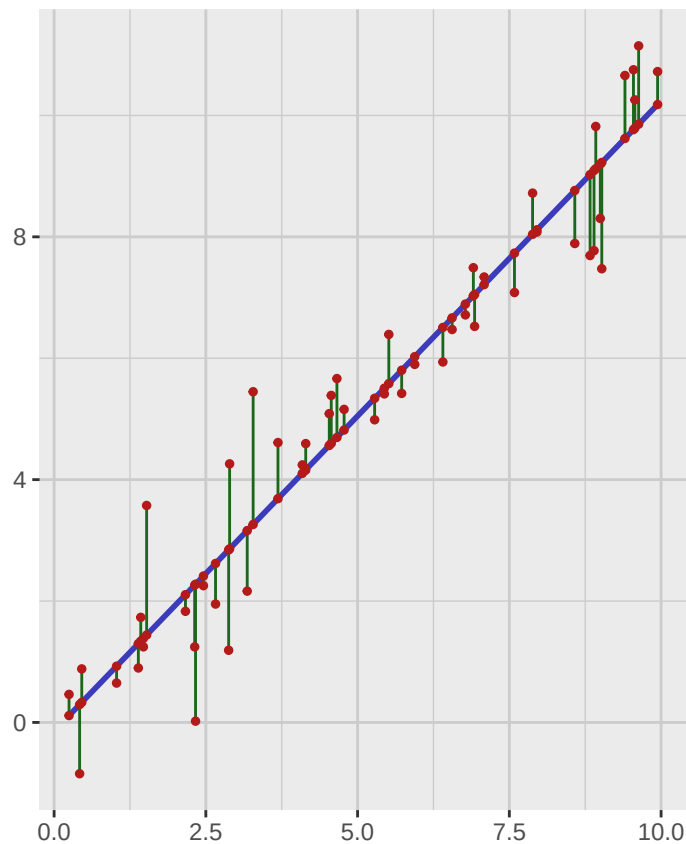


Figura 1.2: Línea recta que mejor se ajusta a los datos simulados.

Notemos que este proceso se puede aplicar a cualquier conjunto de datos dado, sin embargo una de las aplicaciones más comunes de este método es tratar de predecir el valor de la variable Y dado un valor específico de la variable X que no se encuentre en el conjunto de datos original. Se sigue que para que estas predicciones sean de utilidad, es necesario que el comportamiento de nuestros datos siga una tendencia lineal. En caso contrario, las predicciones podrían no ser confiables. Por ejemplo, consideremos el ajuste por el método de mínimos cuadrados para los siguientes conjuntos de datos:

```
library(patchwork)

set.seed(123)

n <- 100
x_data1 <- runif(n, min = 0, max = 10)
epsilon_data1 <- rnorm(n, mean = 0, sd = 10)
y_data1 <- (x_data1)^3 + epsilon_data1

x_data2 <- runif(n, min = 0, max = 10)
epsilon_data2 <- rnorm(n, mean = 0, sd = 0.1)
y_data2 <- sin(x_data2) + epsilon_data2
```

```
x_data3 <- runif(n, min = 0, max = 10)
y_data3 <- runif(n, min = 0, max = 10)

x_data4 <- runif(n, min = 0, max = 10)
y_data4 <- mapply(function(mu, sigma) rnorm(1, mean = mu, sd = sigma),
                  mu = x_data4,
                  sigma = x_data4)

sample_data1_tbl <- tibble(
  x = x_data1,
  y = y_data1
)

sample_data2_tbl <- tibble(
  x = x_data2,
  y = y_data2
)

sample_data3_tbl <- tibble(
  x = x_data3,
  y = y_data3
)

sample_data4_tbl <- tibble(
  x = x_data4,
  y = y_data4
)

sample_data1_plot <- sample_data1_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  labs(
    title = "Datos cúbicos",
    x = "",
    y = ""
  ) +
  theme_krul()

sample_data2_plot <- sample_data2_tbl %>%
  ggplot(aes(x = x, y = y)) +
```

```
geom_point(  
  color = c_pal("C red"),  
  size = 1,  
  alpha = 1  
) +  
geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +  
labs(  
  title = "Datos senoidales",  
  x = "",  
  y = ""  
) +  
theme_krul()  
  
sample_data3_plot <- sample_data3_tbl %>%  
  ggplot(aes(x = x, y = y)) +  
  geom_point(  
    color = c_pal("C red"),  
    size = 1,  
    alpha = 1  
  ) +  
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +  
  labs(  
    title = "Datos aleatorios",  
    x = "",  
    y = ""  
  ) +  
  theme_krul()  
  
sample_data4_plot <- sample_data4_tbl %>%  
  ggplot(aes(x = x, y = y)) +  
  geom_point(  
    color = c_pal("C red"),  
    size = 1,  
    alpha = 1  
  ) +  
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +  
  labs(  
    title = "Datos con varianza creciente",  
    x = "",  
    y = ""  
  ) +  
  theme_krul()  
  
sample_data_plot <- (sample_data1_plot + sample_data2_plot) /  
  (sample_data3_plot + sample_data4_plot)
```

```
sample_data_plot
```

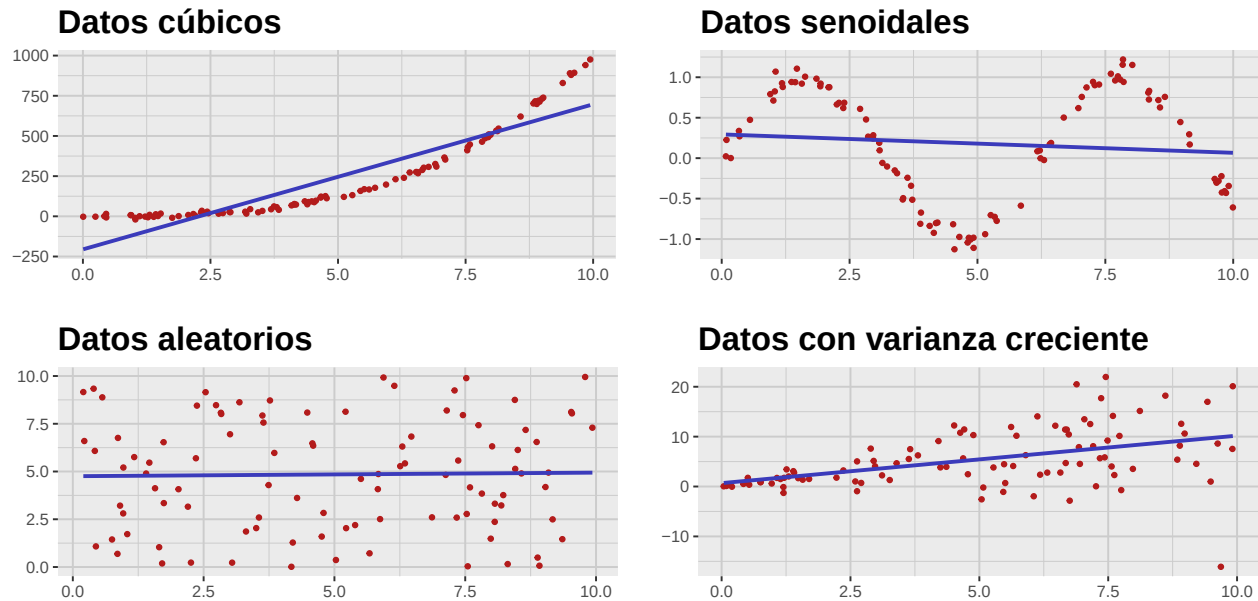


Figura 1.3: Ajuste por mínimos cuadrados para diferentes conjuntos de datos.

Es bastante claro que en estos casos el ajuste por mínimos cuadrados no es el método adecuado para realizar predicciones confiables. En general, este método solo es efectivo cuando se cumplen ciertas hipótesis que estudiaremos en las siguientes secciones, cuando introduciremos el modelo de regresión lineal desde un punto de vista estadístico.